SALESDB PRACTICE SET 2 • T-SQL

Intermediate SQL Concepts Guide

A concept-by-concept walkthrough of every technique used in the 50-task practice set: joins with aggregation, window functions, CTEs, PIVOT, correlated subqueries, string/date logic, data quality checks, and temp tables & stored procedures.

Database: SalesDB (Sales.Customers, Sales.Employees, Sales.Products, Sales.Orders, Sales.OrdersArchive)

Dialect: SQL Server (T-SQL)

Level: Intermediate

WHAT'S INSIDE

- Joins + Aggregation
- Window Functions
- CTEs & Recursion
- PIVOT / UNPIVOT
- Correlated Subqueries
- String + Date Logic
- Data Quality Checks
- Temp Tables & Procedures

1 Joins with Aggregation

> A join by itself just widens a row. The intermediate skill is combining a join with `GROUP BY` to turn transactional rows (one per order) into a business-level summary (one per customer, category, or salesperson).

The concept

When you join `Sales.Orders` to a lookup table like `Sales.Products`, each order row gets duplicated attributes (Category, Product name) attached to it. If you then `GROUP BY` those attributes, SQL Server collapses the many order rows back down into one row per group — but now every aggregate (`SUM`, `COUNT`, `AVG`) is calculated across the joined, enriched dataset.

The rule of thumb: **join first to bring in the descriptive columns you need, group second to collapse to the grain you want.** Anything in the SELECT list that isn't inside an aggregate function must appear in the GROUP BY.

EXAMPLE — TASK 1

Total revenue, quantity, and distinct buyers per product category:

T-SQL

```
SELECT p.Category,
       SUM(o.Sales)                  AS TotalRevenue,
       SUM(o.Quantity)               AS TotalQuantity,
       COUNT(DISTINCT o.CustomerID)  AS DistinctCustomers
FROM Sales.Orders o
INNER JOIN Sales.Products p ON o.ProductID = p.ProductID
GROUP BY p.Category
ORDER BY TotalRevenue DESC;
```

Category	TotalRevenue	TotalQuantity	DistinctCustomers
Electronics	18,420	62	21
Home	15,110	58	19
Sports	12,780	51	17

Also used in Tasks 2, 4, 5, 6, 7, 8

> **TIP**

`COUNT(DISTINCT ...)` is what separates "number of orders" from "number of unique customers" — without DISTINCT, a customer who ordered 5 times would be counted 5 times.

INTERVIEW ANGLE

Interviewers love asking "why did this join duplicate my aggregate?" The answer is almost always: you joined to a table with a one-to-many relationship (e.g., Orders → OrderItems) before aggregating, inflating SUM(). Always check cardinality before you GROUP BY.

2 Window Functions (Advanced)

> Window functions calculate across a set of rows *related to the current row* without collapsing them into one row — unlike `GROUP BY`, every input row survives in the output.

The concept

Every window function follows the same shape: `FUNCTION() OVER (PARTITION BY ... ORDER BY ... [frame])`.

- **PARTITION BY** — resets the calculation for each group (like a GROUP BY that doesn't collapse rows).
- **ORDER BY** — defines the sequence used for ranking, running totals, or LAG/LEAD.
- **Frame** (`ROWS BETWEEN ...`) — narrows the window to a sliding range, e.g. "the current row and the 2 before it" for a moving average.

EXAMPLE — TASK 16: BIGGEST SALES DROP PER SALESPERSON

T-SQL

```
WITH OrderChanges AS (
    SELECT OrderID, SalesPersonID, OrderDate, Sales,
           Sales - LAG(Sales) OVER (PARTITION BY SalesPersonID ORDER BY OrderDate) AS
SalesChange
    FROM Sales.Orders
),
RankedDrops AS (
    SELECT *, RANK() OVER (PARTITION BY SalesPersonID ORDER BY SalesChange ASC) AS
DropRank
    FROM OrderChanges
    WHERE SalesChange IS NOT NULL
)
SELECT OrderID, SalesPersonID, OrderDate, Sales, SalesChange
FROM RankedDrops
WHERE DropRank = 1;
```

`LAG(Sales)` looks one row back *within the same SalesPersonID partition*, so it never accidentally compares two different salespeople. Subtracting gives the change; ranking the changes ascending puts the worst drop first.

RANKING FUNCTIONS COMPARED

Function	Ties	Gaps after a tie	Typical use

`ROW_NUMBER()`	Breaks ties arbitrarily	Never	"Give me exactly 1 row per group" (Task 49/50-style de-dup)
`RANK()`	Same rank for ties	Yes (1,1,3)	Leaderboards where ties matter
`DENSE_RANK()`	Same rank for ties	No (1,1,2)	"Top N distinct values" (Task 10)

TIP

`PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY Sales) OVER ()` (Task 12) is the T-SQL way to get a true statistical median — `AVG()` is not the same thing once your data has outliers.

3 CTEs & Recursion

> A Common Table Expression (`WITH name AS (...)`) is a named, temporary result set that exists only for the duration of the query that follows it.

Why use a CTE instead of a subquery?

Functionally, a simple CTE and a derived subquery `(SELECT ... ) x` are equivalent. The value of a CTE is readability and chaining: you can define `CTE_A`, then build `CTE_B` on top of it, and reference `CTE_A` again later — all without nesting nine levels of parentheses.

EXAMPLE — TASK 20: CHAINED CTES

T-SQL

```
WITH CategorySpend AS (
    SELECT o.CustomerID, p.Category, SUM(o.Sales) AS Spend
    FROM Sales.Orders o
    INNER JOIN Sales.Products p ON o.ProductID = p.ProductID
    GROUP BY o.CustomerID, p.Category
),
RankedCategorySpend AS (
    SELECT *, RANK() OVER (PARTITION BY Category ORDER BY Spend DESC) AS rnk
    FROM CategorySpend
)
SELECT Category, CustomerID, Spend
FROM RankedCategorySpend
WHERE rnk = 1;
```

Recursive CTEs

A recursive CTE has two parts joined by `UNION ALL`: an **anchor member** (the starting rows) and a **recursive member** that joins the CTE back to itself, repeating until no new rows are produced. This is the standard pattern for hierarchies (org charts, bill-of-materials) and generated sequences (calendars, number spines).

EXAMPLE — TASK 19: MANAGER CHAIN

T-SQL

```
WITH ManagerChain AS (
    -- Anchor: the starting employee
    SELECT EmployeeID, FirstName, LastName, ManagerID, 0 AS ChainLevel
    FROM Sales.Employees
    WHERE EmployeeID = 25
```

```
    UNION ALL
    -- Recursive: climb one manager level at a time
    SELECT e.EmployeeID, e.FirstName, e.LastName, e.ManagerID, mc.ChainLevel + 1
    FROM Sales.Employees e
    INNER JOIN ManagerChain mc ON e.EmployeeID = mc.ManagerID
)
SELECT * FROM ManagerChain ORDER BY ChainLevel;
```

WATCH OUT

Recursive CTEs default to a 100-iteration safety limit. If your hierarchy or date spine needs more, add `OPTION (MAXRECURSION 1000)` (or `0` for unlimited) as shown in Tasks 23 and 70 — but always keep a stop condition in the recursive member, or you'll spin forever.

4 PIVOT & Conditional Aggregation

> Both techniques turn *rows* of category values into *columns*. PIVOT is T-SQL's dedicated syntax; conditional aggregation (`SUM(CASE WHEN ..)`) does the same job with plain SQL and works on every database engine.

PIVOT SYNTAX (TASK 25)

T-SQL

```
SELECT OrderYear, Electronics, Home,
Sports
FROM ( SELECT YEAR(o.OrderDate)
OrderYear,
         p.Category, o.Sales
  FROM Sales.Orders o
  JOIN Sales.Products p
    ON o.ProductID=p.ProductID ) src
PIVOT (
  SUM(Sales) FOR Category
  IN ([Electronics],[Home],[Sports])
) piv;
```

CONDITIONAL AGGREGATION (TASK 27)

T-SQL

```
SELECT YEAR(OrderDate) OrderYear,
  SUM(CASE WHEN OrderStatus=
    'Delivered' THEN 1 ELSE 0 END)
    AS Delivered,
  SUM(CASE WHEN OrderStatus=
    'Shipped' THEN 1 ELSE 0 END)
    AS Shipped
FROM Sales.Orders
GROUP BY YEAR(OrderDate);
```

	PIVOT	Conditional Aggregation
Column list	Must be known and listed explicitly	Also fixed, but reads more like plain SQL
Portability	SQL Server / Oracle specific syntax	Works on MySQL, Postgres, SQLite, etc.
Multiple aggregates per column	Awkward (one aggregate per PIVOT)	Easy — just add more CASE expressions

UNPIVOT

UNPIVOT (Task 28) reverses the operation: it takes several columns (Accessories, Clothing, Electronics...) and stacks them back into two columns — a label column and a value column. It's the SQL equivalent of "melting" a wide table into a long one, useful before charting or loading into a fact table.

> **INTERVIEW ANGLE**
>
> Interviewers often ask you to build a pivot without the PIVOT keyword to test whether you understand `CASE` + `GROUP BY` from first principles — that's exactly what conditional aggregation demonstrates.

5 Correlated & Nested Subqueries

> A **nested** subquery runs once, independently, and its result feeds the outer query. A **correlated** subquery references a column from the outer query, so conceptually it re-runs once per outer row.

EXAMPLE — TASK 31: CORRELATED SUBQUERY

T-SQL

```
SELECT e1.EmployeeID, e1.Department, e1.Salary
FROM Sales.Employees e1
WHERE e1.Salary > (
    SELECT AVG(e2.Salary)
    FROM Sales.Employees e2
    WHERE e2.Department = e1.Department    -- correlation
);
```

The inner query's `WHERE e2.Department = e1.Department` ties it to the current outer row (`e1`), so the average is recalculated per department rather than once for the whole table.

EXAMPLE — TASK 34: NESTED (INDEPENDENT) SUBQUERY

T-SQL

```
SELECT CustomerID, TotalSpend
FROM ( SELECT CustomerID, SUM(Sales) TotalSpend
       FROM Sales.Orders GROUP BY CustomerID ) spend
WHERE TotalSpend > (
    SELECT AVG(TotalSpend)
    FROM ( SELECT SUM(Sales) TotalSpend
           FROM Sales.Orders GROUP BY CustomerID ) avg_sub
);
```

The "for all" pattern — Task 35

SQL has no direct "for every row" operator, so the standard trick is a double negative: `EXISTS` (at least one order exists) combined with `NOT EXISTS` (no order violates the condition). Together they mean "this salesperson has orders, and none of them are ≤ 100" — i.e. *all* of their orders are above 100.

T-SQL

```
WHERE EXISTS (SELECT 1 FROM Sales.Orders o WHERE o.SalesPersonID = e.EmployeeID)
  AND NOT EXISTS (
      SELECT 1 FROM Sales.Orders o
```

```
    WHERE o.SalesPersonID = e.EmployeeID AND o.Sales <= 100
);
```

TIP

`EXISTS` is usually faster than `IN` for large tables because the optimizer can stop at the first match instead of building the full result set of the subquery.

6 String + Date Combinations

> Real reporting tasks rarely need a string function or a date function alone — they need both, combined into one formatted, human-readable field.

EXAMPLE — TASK 37: PADDED IDS + FORMATTED DATES

```T-SQL
SELECT OrderID,
   'ORD-' + RIGHT('000000' + CAST(OrderID AS VARCHAR(10)), 6)
   + ' | ' + FORMAT(OrderDate, 'yyyy-MM-dd') AS OrderLabel
FROM Sales.Orders;
```

OrderID	OrderLabel
12	ORD-000012 | 2024-06-12
105	ORD-000105 | 2024-02-25

`RIGHT('000000' + CAST(...), 6)` is the classic zero-padding trick: pad the number with far more leading zeros than you'll ever need, then take the rightmost N characters.

Key functions used across Tasks 37-42

Function	Purpose	Example
`FORMAT(date, pattern)`	Human-readable date/number formatting	`FORMAT(OrderDate,'MMM yyyy')` → "Aug 2024"
`DATENAME(part, date)`	Text name of a date part	`DATENAME(WEEKDAY, OrderDate)` → "Tuesday"
`DATEPART(part, date)`	Numeric value of a date part (sortable)	`DATEPART(WEEKDAY, OrderDate)` → 3
`LEFT / RIGHT`	Substring from either end	`LEFT(FirstName,1)` → initial
`ISNULL(val, default)`	Fallback text for missing values	`ISNULL(ShipAddress,'N/A')`

WATCH OUT

> Use `DATENAME` for display but `DATEPART` for sorting (Task 39) — sorting alphabetically by weekday name puts "Friday" before "Monday", which is wrong; sorting by the numeric `DATEPART` keeps the week in order.

7 Data Quality & Anomaly Detection

> Before trusting any report, a data engineer checks whether the source data is internally consistent. These queries don't answer a business question directly — they answer "can I trust this table?"

EXAMPLE — TASK 43: RECOMPUTE AND COMPARE

T-SQL

```
SELECT o.OrderID, o.Quantity, p.Price,
       o.Quantity * p.Price AS ExpectedSales,
       o.Sales              AS ActualSales
FROM Sales.Orders o
INNER JOIN Sales.Products p ON o.ProductID = p.ProductID
WHERE o.Sales <> o.Quantity * p.Price;
```

The pattern is always the same: derive what a value *should* be from other columns, then compare it to the stored value and surface the mismatches.

EXAMPLE — TASK 45: REFERENTIAL INTEGRITY WITHOUT FOREIGN KEYS

T-SQL

```
SELECT o.OrderID, 'Invalid CustomerID' AS Issue
FROM Sales.Orders o
LEFT JOIN Sales.Customers c ON o.CustomerID = c.CustomerID
WHERE c.CustomerID IS NULL
UNION ALL
-- ...repeated for ProductID and SalesPersonID
```

A `LEFT JOIN` followed by `WHERE right_table.key IS NULL` is the universal "find orphaned rows" pattern — it works whether or not the database actually enforces a foreign key constraint.

The four checks in this set

- **Recomputation checks** (Task 43) — does a derived value match the stored value?
- **Duplicate detection** (Task 44) — `GROUP BY` the natural key, `HAVING COUNT(*) > 1`.
- **Referential integrity** (Task 45) — LEFT JOIN + IS NULL to find orphans.
- **Logical consistency** (Task 46) — e.g. `ShipDate < OrderDate`, a date that violates real-world sequencing.

> **INTERVIEW ANGLE**

> Data quality queries are a favorite in data engineering interviews because they test whether you think about a dataset critically instead of just querying it at face value.

8 Temp Tables & Stored Procedures

> Once a query needs to be reused, staged, or parameterized, it graduates from a single `SELECT` into temp tables, table variables, or stored procedures.

TEMP TABLE (TASK 47)

T-SQL

```
SELECT TOP 10 CustomerID, SUM(Sales) AS TotalSpend
INTO #TopCustomers
FROM Sales.Orders
GROUP BY CustomerID
ORDER BY TotalSpend DESC;

SELECT c.*, t.TotalSpend
FROM #TopCustomers t
INNER JOIN Sales.Customers c ON t.CustomerID = c.CustomerID;

DROP TABLE #TopCustomers;
```

`SELECT ... INTO #Table` materializes results into `tempdb` — useful when the same intermediate result will be joined against multiple times, since SQL Server only computes it once.

TABLE VARIABLE (TASK 48)

A `DECLARE @Table TABLE (...)` behaves like a temp table but lives in memory for the scope of the batch. It's lighter weight for small, short-lived staging data, though it doesn't support indexes or statistics the way a real temp table does.

STORED PROCEDURE (TASK 49)

T-SQL

```
CREATE OR ALTER PROCEDURE Sales.usp_GetCustomerOrders
    @CustomerID INT
AS
BEGIN
    SET NOCOUNT ON;
    SELECT o.OrderID, o.OrderDate, o.Sales, p.Product
    FROM Sales.Orders o
    INNER JOIN Sales.Products p ON o.ProductID = p.ProductID
    WHERE o.CustomerID = @CustomerID;
END;
```

```
EXEC Sales.usp_GetCustomerOrders @CustomerID = 5;
```

A stored procedure packages a parameterized query as a reusable, named object. Compared to a raw ad-hoc query, it gives you a stable execution plan, centralized permissions, and a single point of maintenance if the logic changes.

	Temp table (#T)	Table variable (@T)	Stored procedure
Scope	Session	Batch	Reusable, database object
Indexes	Yes	Limited	N/A
Best for	Larger staged datasets	Small lookup/staging sets	Parameterized, reusable logic

TIP

`SET NOCOUNT ON` at the top of a procedure suppresses the "(N rows affected)" message, which reduces network chatter and is considered standard practice in production stored procedures.